DATA ANALYTICS CASE STUDY

Dubai Smart Traffic Intelligence Platform

End-to-end analytics on 700K+ UAE traffic incidents. From raw open data through SQL ETL, dimensional modelling, and an executive Power BI dashboard.

STACK SQL Server • Power BI • DAX • Power Query • Star Schema Modelling

705,944	667	96.9%	2018-26
Final Clean Rows	Exact Duplicates Removed	Valid Geo Coverage	Years Analysed

Contents

1. Executive Summary

This project transforms a noisy, multilingual UAE open-data traffic feed into a production-style analytics platform. The work spans the full data lifecycle: ingestion, encoding repair, deduplication, geospatial validation, dimensional modelling, and an interactive Power BI dashboard built for non-technical decision-makers.

What the dashboard answers

- Where are the highest-density traffic incident zones in Dubai?
- When do incidents peak, by hour, day, month, and year?
- Which incident categories dominate operationally?
- How did 2020 movement restrictions reshape incident volumes?

Headline findings

- 705,944 cleaned incidents across 2018-2026, with 233 distinct incident categories.
- Citywide incidents peak at 2 PM, but several hotspot zones concentrate between 5-9 PM. Global patterns and local patterns tell different stories.
- Thursday is the busiest day; July-August are the quietest months.
- 96.9% of records carry valid UAE coordinates after geospatial validation.

> **KEY INSIGHT**
>
> The most interesting result emerged only when temporal and spatial analysis were combined. The city's overall 2 PM peak fragments at the zone level and zones around 25.20° / 55.28° peak between 5-9 PM. This directly shaped the dashboard's interactive filter design.

2. Project Scope & Dataset

The platform simulates a real-world traffic intelligence solution supporting road safety analysis, smart city initiatives, operational monitoring, and trend forecasting. Every design choice, from NVARCHAR encoding to layered cleaning tables, mirrors patterns used by enterprise analytics teams.

Goals

- Identify accident patterns and high-risk incident types across Dubai.
- Detect and quantify data quality issues - duplicates, NULLs, invalid coordinates.
- Build a clean dimensional model suitable for BI reporting.
- Deliver an executive dashboard with both temporal and spatial insight.

Dataset overview

Attribute	Detail
Source	Dubai Pulse - Traffic Incidents Open Dataset
Raw row count	~709,838
Time span	2018 → 2026
Key fields	accident ID, timestamp, type (Arabic), latitude, longitude, load timestamp
Format	CSV (UTF-8 corrupted) → converted to .xlsx

3. Data Ingestion & Encoding

Problem: corrupted Arabic text

When opened directly, the raw CSV displayed Arabic incident descriptions as mojibake (e.g. Øµ Ø¯Ù...) due to a UTF-8 / Windows-default mismatch on CSV import.

Resolution

- Re-saved the CSV as Excel (.xlsx) to preserve native Arabic encoding before SQL Server ingestion.
- Used NVARCHAR(MAX) instead of VARCHAR for all text columns, a deliberate multilingual database design choice.
- Allowed NULL on acci_x, acci_y, and acci_name to handle realistic incomplete records.

Errors encountered and resolved

Error	Cause	Resolution
Object 'Raw_Traffic_Incidents' already exists	Partial prior import	Created Raw_Traffic_Incidents_v2 to preserve continuity
acci_x does not allow DBNull.Value	Missing coordinates in source	Made coordinate columns nullable
Arabic text garbled	UTF-8 / Windows CSV mismatch	Converted source to .xlsx and used NVARCHAR(MAX)

4. Data Quality Investigation

Before any cleaning, I ran a full integrity audit. Three findings shaped the entire cleaning strategy.

Finding 1 - Duplicate accident IDs

Some accident IDs appeared dozens of times. One ID, 3377595397, appeared 73 times. Initial hypothesis: ETL ingestion duplication or system update logging.

```
SELECT acci_id, COUNT(*) AS duplicate_count
FROM Raw_Traffic_Incidents_v2
GROUP BY acci_id
HAVING COUNT(*) > 1
ORDER BY duplicate_count DESC;
```

Finding 2 - True identical duplicate rows

A single identical record appeared 54 times, same ID, same coordinates, same timestamp. Confirmed ingestion-level duplication, not just versioning.

Finding 3 - Partial-record duplicates

Some duplicates were not identical, one row had NULL coordinates, another row for the same incident had valid coordinates. This suggested update-history behaviour where coordinates were enriched over time.

> **WHY THIS MATTERS**
>
> Don't remove duplicates blindly. The data told me to preserve the highest-quality version of each incident, which became the basis of the rank-based deduplication strategy in Section 5.

5. Multi-Stage Cleaning Pipeline

Layered cleaning following a medallion-style architecture: raw → cleaned → analytical. The raw layer is preserved as an immutable baseline.

Stage 1 - Remove exact duplicates

```
SELECT DISTINCT *
INTO clean_traffic_incidents
FROM Raw_Traffic_Incidents_v2;
```

Result: 667 fully identical rows removed.

Stage 2 - Rank-based deduplication for partial duplicates

Used ROW_NUMBER() with a quality-priority CASE expression to keep the best version of each accident:

```
WITH ranked AS (
  SELECT *,
    ROW_NUMBER() OVER (
      PARTITION BY acci_id
      ORDER BY
        CASE WHEN acci_x IS NOT NULL AND acci_y IS NOT NULL THEN 1 ELSE 2 END,
        load_timestamp DESC
    ) AS rn
  FROM clean_traffic_incidents
)
SELECT *
INTO Final_Clean_Traffic_Incidents
FROM ranked
WHERE rn = 1;
```

Priority logic

- PARTITION BY acci_id - ranking restarts per incident.
- CASE WHEN - rows with valid coordinates win over rows with NULLs.
- load_timestamp DESC - when multiple valid rows exist, the most recent wins. This mirrors the enterprise pattern of "latest valid record."

Validation

Post-cleaning duplicate check returned **zero duplicate accident IDs**. One optimized record per incident, confirmed clean.

6. Exploratory Data Analysis

With the data clean, I ran five analytical lenses to surface patterns worth visualizing.

6.1 Annual trends

- Textbook COVID-19 dip in 2020, steady recovery through 2025.
- 2018 and 2026 appear as partial-year extractions, direct year-over-year comparisons need normalization.

6.2 Incident type breakdown

Top categories reveal the dataset's true nature, operational traffic data, not severe accidents only:

- Vehicle breakdowns
- Double parking incidents
- Collisions with poles, walls, and barriers
- Low-speed urban collisions

6.3 Monthly seasonality

- High: November, December, January, February (cooler weather, tourism peak).
- Low: July, August (summer heat, reduced outdoor mobility).

6.4 Day-of-week patterns

- Highest: Thursday, Wednesday, Tuesday.
- Lowest: Friday.

6.5 Hour-of-day patterns

- Peak window: 12 PM-4 PM citywide.
- Trough: 2 AM-5 AM.

6.6 Combined day × hour

Strongest concentration window: Thursday 1 PM-6 PM. Weekday afternoons consistently dominate.

7. Geospatial Validation & Hotspots

The problem

Initial coordinate inspection revealed values far outside UAE geographic bounds, entries like 102.x, 110.x, 131.x. Possible causes: mixed coordinate systems, corrupted GIS records, or upstream entry errors.

UAE coordinate validation

Field	Valid UAE Range
Latitude (acci_y)	22 → 27
Longitude (acci_x)	51 → 57

```
SELECT COUNT(*) AS valid_uae_records
FROM Final_Clean_Traffic_Incidents
WHERE acci_y BETWEEN 22 AND 27
  AND acci_x BETWEEN 51 AND 57;
```

Result: 684,113 valid UAE records (~96.9%). The remaining 3.1% was isolated into a separate audit layer rather than discarded, operational events should not be lost.

Hotspot aggregation

Rounded coordinates to 2 decimal places to cluster nearby GPS points into operational zones:

```
SELECT
  ROUND(acci_y, 2) AS lat_zone,
  ROUND(acci_x, 2) AS lng_zone,
  COUNT(*)         AS incident_count
FROM Final_Clean_Traffic_Incidents
WHERE acci_y BETWEEN 22 AND 27 AND acci_x BETWEEN 51 AND 57
GROUP BY ROUND(acci_y, 2), ROUND(acci_x, 2)
ORDER BY incident_count DESC;
```

Top hotspot zones

Zone (lat / lng)	Incident Volume
25.20 / 55.28	6,056
25.23 / 55.29	5,618
25.08 / 55.14	5,273

THE MOST INTERESTING FINDING

When I combined geospatial and temporal analysis, the city's overall 2 PM peak fragmented. Several hotspot zones, particularly around 25.20° / 55.28°, concentrated their activity between 5-9 PM instead. Local behaviour diverged from global behaviour. This insight only emerges when you cross-cut dimensions.

8. Dimensional Modelling (Star Schema)

To support fast, flexible BI reporting, I restructured the flat dataset into a star schema. Dimension tables sit on the "one" side; the fact table sits on the "many" side.

Fact table - Fact_Traffic_Incidents

Column	Purpose
incident_id	Unique incident key
incident_datetime	Exact event timestamp
incident_date	Foreign key → Dim_Date
incident_hour	Hourly aggregation
incident_type_key	Foreign key → Dim_Incident_Type
latitude / longitude	Raw coordinates
latitude_zone / longitude_zone	Foreign keys → Dim_Geo_Zones
load_timestamp	Ingestion tracking

Fact rows: 705,944, matches the cleaned operational dataset.

Dimensions

Dimension	Rows	Purpose
Dim_Date	2,660	Year, month, quarter, weekday, time intelligence
Dim_Incident_Type	233	Operational category filtering
Dim_Geo_Zones	2,374	Rounded coordinate clusters for hotspot analysis

Final schema

```
         Dim_Date
             \
              \
Dim_Incident_Type ── Fact_Traffic_Incidents ── Dim_Geo_Zones
              /
             /
        (foreign keys)
```

This structure delivers fast slicing on date, category, and geography simultaneously, the foundation for every visual in the dashboard.

9. Power BI Implementation

With the star schema ready, I built the dashboard in Power BI Desktop. The design priority was clarity for non-technical viewers: KPI cards on top, trend and category context in the middle, geospatial and weekly/monthly breakdowns on the bottom row.

9.1 Connecting to SQL Server

- Home → Get Data → SQL Server.
- Server: local SQL Server instance. Database: UAE_TRAFFIC.
- Mode: Import (chosen for faster filter response over DirectQuery).

Tables loaded

- Fact_Traffic_Incidents
- Dim_Date
- Dim_Incident_Type
- Dim_Geo_Zones
- Geo_Clean_Traffic_Incidents

Tables intentionally excluded

- Raw_Traffic_Incidents_v2
- clean_traffic_incidents
- Final_Clean_Traffic_Incidents

These were SQL preparation tables. Loading them would bloat the model and confuse the relationship view, only the final analytical layer belongs in Power BI.

9.2 Relationships (Star Schema in Power BI)

Date relationship

```
Dim_Date[full_date]   →   Fact_Traffic_Incidents[incident_date]
Cardinality: One-to-Many
Cross-filter direction: Single
```

Incident Type relationship

```
Dim_Incident_Type[incident_type]   →   Fact_Traffic_Incidents[incident_type_key]
Cardinality: One-to-Many
Cross-filter direction: Single
```

Why this matters

Dimension tables on the "one" side, fact on the "many" side, textbook star schema. When the user clicks Thursday in a slicer, the filter automatically propagates from Dim_Date into every related row in Fact_Traffic_Incidents, which in turn re-aggregates every visual on the page.

> **ONE → MANY**
>
> One date can relate to thousands of incidents. One incident category can relate to thousands of incidents. That's the cardinality, and it's what makes filtering performant even on a 705K-row fact table.

9.3 KPI Cards

Four headline KPIs anchor the top of the dashboard. The first uses a simple count; the rest use DAX measures so they recalculate correctly under every slicer combination.

KPI 1 - Total Incidents

Drag incident_id onto a card visual and set the aggregation to Count.

Result: 705,944

KPI 2 - Peak Hour

DAX measure that groups incidents by hour, ranks them, and returns the hour with the highest count:

```
Peak Hour =
VAR HourTable =
    SUMMARIZE(
        Fact_Traffic_Incidents,
        Fact_Traffic_Incidents[incident_hour],
        "@count", COUNT( Fact_Traffic_Incidents[incident_id] )
    )
RETURN
    MAXX(
        TOPN( 1, HourTable, [@count], DESC ),
        Fact_Traffic_Incidents[incident_hour]
    )
```

Raw output: 14 (24-hour format). To display as a human-readable "2 PM", I wrapped the hour in a formatting measure:

```
Peak Hour (Formatted) =
VAR h = [Peak Hour]
RETURN
    IF( h = 0,  "12 AM",
    IF( h < 12, FORMAT( h, "0" ) & " AM",
    IF( h = 12, "12 PM",
                FORMAT( h - 12, "0" ) & " PM" )))
```

KPI 3 - Busiest Day

```
Busiest Day =
VAR DayTable =
    SUMMARIZE(
        Fact_Traffic_Incidents,
        Dim_Date[weekday_name],
        "@count", COUNT( Fact_Traffic_Incidents[incident_id] )
    )
RETURN
```

```
    MAXX(
        TOPN( 1, DayTable, [@count], DESC ),
        Dim_Date[weekday_name]
    )
```

Result: Thursday

KPI 4 - Geo Coverage %

```
Geo Coverage % =
DIVIDE(
    CALCULATE(
        COUNT( Fact_Traffic_Incidents[incident_id] ),
        Fact_Traffic_Incidents[latitude]  >= 22,
        Fact_Traffic_Incidents[latitude]  <= 27,
        Fact_Traffic_Incidents[longitude] >= 51,
        Fact_Traffic_Incidents[longitude] <= 57
    ),
    COUNT( Fact_Traffic_Incidents[incident_id] )
)
```

Result: 96.9%

9.4 Visuals

Six core visuals plus two slicers, arranged in a single-page executive layout on a dark Dubai-skyline background.

Visual	Type	Configuration
Traffic Incidents Over Time	Line chart	X: Dim_Date[year] | Y: Count of incident_id
Traffic Incidents by Month	Clustered column	X: Dim_Date[month_name] | Y: Count of incident_id
Traffic Incidents by Weekday	Horizontal bar	Y: Dim_Date[weekday_name] | X: Count of incident_id
Top Incident Types	Horizontal bar	Y: Dim_Incident_Type[incident_type] | X: Count of incident_id | Top N = 10
Traffic Incident Hotspots	Map (bubble)	Latitude: acci_x | Longitude: acci_y | Size: Count of incident_id
Year slicer	Slicer (dropdown)	Field: Dim_Date[year]
Incident Filter slicer	Slicer (dropdown)	Field: Dim_Incident_Type[incident_type]

Why these choices

- Top N = 10 on the incident type chart prevents 233 categories from flooding the visual.
- Dropdown slicers (Slicer Settings → Style → Dropdown) save space and look more professional than a long list.
- Native Arabic labels were preserved on the incident type bar chart, proves the encoding work paid off and serves UAE stakeholders authentically.
- Map uses bubble density rather than choropleth, better suited to point-incident data.
- Sorted weekday_name by weekday_number and month_name by month_number so charts display in correct chronological order, not alphabetical.

9.5 Interactivity

- Click any year on the line chart → every other visual cross-filters to that year.
- Click any weekday → the map, monthly chart, and incident-type chart re-aggregate.
- Click any incident type → the map shows only that category's geographic distribution, surfacing category-specific hotspots.

10. Outcomes & Business Value

Quantitative outcomes

Metric	Result
Final clean fact rows	705,944
Exact duplicates removed	667
Distinct incident categories captured	233
Geo coverage after validation	96.9% (684,113 valid records)
Unique operational dates modelled	2,660
Hotspot zones identified	2,374

Business value

- Road safety prioritization, identifies the streets and times where intervention has the highest impact.
- Operational planning, staffing and resource allocation tied to verified peak windows.
- Data quality benchmarking, explicit 96.9% geo coverage gives stakeholders a transparent confidence metric.
- Reusable foundation, the star schema supports future modules (severity scoring, weather joins, predictive modelling) without restructure.

Skills demonstrated

- SQL Server administration, window functions, conditional aggregation, layered ETL design
- Multilingual database design (NVARCHAR, encoding repair)
- Data quality investigation and quality-aware deduplication
- Geospatial validation and zone-based aggregation
- Dimensional modelling (star schema, fact + dimension design)
- Power BI: relationships, DAX (SUMMARIZE, TOPN, MAXX, CALCULATE, DIVIDE, FORMAT), Power Query, executive dashboard design
- Translating ambiguous operational data into clear stakeholder narratives

11. Roadmap

- Severity classification model, train a supervised model on incident type + location to estimate severity tiers.
- Weather data join, overlay UAE weather records to test correlation with seasonal spikes.
- Forecasting layer, Power BI's built-in forecast or Python integration for short-horizon volume forecasts.
- Drill-through pages, dedicated zone-detail and category-detail pages for analyst use.
- Row-level security, if extended to multi-tenant use (e.g. per-municipality filtering).

This project is intentionally scoped as a portfolio case study, but every architectural choice immutable raw layer, layered cleaning, dimensional modelling, quality-aware deduplication maps directly onto how production analytics team’s work.